KernelLens: Compiling PyTorch Triton Kernels into C++ Inference Plugins

Justin Duc *justin.duc@epitech.eu*
Galiad Research

Abstract

Writing custom GPU kernels in OpenAI Triton [1] has become routine; *deploying* them is not. Production inference is increasingly served by pure C++ engines—Microsoft ONNX Runtime (ORT) [12] and NVIDIA TensorRT (TRT) [13]—which apply whole-graph optimisation, engine tuning and VRAM planning. Yet the standard export path, `torch.onnx.export`, cannot trace an imperative `@triton.jit` call, and a hand-written C++ plugin costs thousands of lines per kernel.

We present **KernelLens**, a compiler and runtime that closes this gap through four phases: (1) *dual-pass tracing and static AST analysis* that recover the compiled PTX, the dynamic launch grid and a per-argument input/output/in-place taxonomy; (2) *ONNX graph rewriting* that injects custom `triton_custom` nodes and binds host scalars as CPU inputs; (3) *automated plugin synthesis*, producing `OrtCustomOp` and `nvinfer1::IPluginV2DynamicExt` C++ sources with PTX entry-slot ABI reconstruction and PTX ISA version clamping for Blackwell; and (4) *zero-copy VRAM binding* that registers PyTorch `data_ptr()` addresses directly with each engine.

We evaluate KernelLens on research kernels from LLaMA 3 [2], PaLM, Qwen 2.5 [4] and Liger-Kernel [3], on FP8 and INT4 quantized GEMMs, and on a 4-layer LLaMA decoder, all measured on an RTX 5060 Ti (Blackwell, `sm_120a`). KernelLens reproduces native Triton execution exactly (MaxDiff = 0.00e+00) on the four research kernels inside ONNX Runtime, and its TensorRT plugins are up to 3.3× faster than `torch.compile` on custom attention while attaining the lowest peak VRAM. We also report in full the TensorRT paths that remain incorrect, document the toolchain bugs found during verification, and explicitly correct several claims made in earlier drafts of this work.

1 Introduction

Deep learning systems rely on domain-specific GPU kernels to fuse multi-stage attention, normalisation and activation blocks, reducing both VRAM bandwidth demand and launch overhead. OpenAI Triton is a standard programming model for authoring such kernels in Python, exposing CUDA-like execution control with a Pythonic surface.

A friction point appears at deployment. Enterprise inference pipelines increasingly execute models inside C++ engines—ONNX Runtime and TensorRT—to obtain low latency, stream concurrency, hardware-specific kernel selection and global memory planning. A PyTorch model that contains a custom `@triton.jit` kernel does not fit that pipeline cleanly. The exporter (`torch.onnx.export`) operates on TorchScript or TorchDynamo IR that represents native PyTorch C++ operators; an imperative Python call to a `triton.JITFunction` is opaque to it, so export either fails or silently drops the kernel. The developer is left with two unattractive options: keep the model in Python (and forfeit the C++ engine), or re-implement each kernel as a hand-written C++ plugin (schema registration, PTX ABI, launch geometry, build scripts).

1.1 The five gaps

Concretely, a Triton kernel presents five properties that a C++ engine must recover, none of which survive a naive graph export:

1. **PTX entry ABI mismatch.** Triton compiles Python to PTX and emits an `.entry` declaration whose parameters are *re-ordered*, occasionally *dropped*, and fixed to specific bitwidths. The host must rebuild the exact slot layout rather than trust the Python signature.
2. **Dynamic launch grid.** The grid is an arbitrary Python expression over runtime tensor shapes, e.g. `(s0*s1+127)//128`. It cannot be frozen at compile time and must be re-evaluated inside the C++ engine.
3. **Nested code and in-place mutability.** Research kernels delegate stores to nested helpers and frequently mutate a buffer in place ($\text{ptr}_{\text{out}} \equiv \text{ptr}_{\text{in}}$). A static compiler that misses this produces a spurious copy or a corrupted graph.
4. **Dynamic host scalars.** Parameters such as a soft-cap τ or scaling factor α change per forward pass and must be accepted as host-side CPU inputs without recompilation.
5. **Bitwidths and quantized layout.** FP8 and sub-byte INT4 tensors require 1-byte pointer mapping and dtype-accurate memory placement.

1.2 Contributions

KernelLens compiles a `torch.nn.Module` containing custom `@triton.jit` kernels into native C++ artifacts for both engines, with no manual C++ programming. Its contributions are:

1. **Dual-pass tracing** (§4.1) that couples a concrete eager pass—to capture real PTX, shared memory and strides—with a symbolic `make_fx` pass to recover dynamic grid expressions, plus a static AST scan that classifies each argument as input, output or in-place.
2. **Automated graph rewriting** (§4.2) that injects custom ONNX nodes, binds dynamic scalars as CPU inputs and aliases in-place buffers.
3. **Automated plugin synthesis** (§4.3) for both `OrtCustomOp` and `IPluginV2DynamicExt`, including PTX entry-slot ABI reconstruction and PTX ISA version clamping.
4. **Zero-copy runtime binding** (§4.4) that hands PyTorch GPU addresses straight to each engine.
5. **An empirical and verification study** (§6, §7) across LLM research kernels, quantized kernels and a full decoder, together with the toolchain fixes required to make the system work off the vendor's default environment, and a candid account of the paths that still fail.

1.3 Scope

KernelLens targets CUDA C++ engines on NVIDIA hardware. Everything reported in §6 was measured on a single RTX 5060 Ti; we do not extrapolate to clusters or other vendors. Where an earlier draft of this report described a feature that the implementation does not provide, we say so explicitly in §7.

2 Background and Related Work

PyTorch compilers. PyTorch 2.0's `torch.compile`, powered by TorchInductor [8], generates Triton for eager PyTorch graphs and is the natural performance baseline for custom kernels. It optimises execution *inside* a Python process and depends on the `libtorch` runtime; it does not emit a standalone ONNX graph or a C++ plugin suitable for a non-Python serving process. AOTInductor [9] compiles graphs to shared libraries but still links against PyTorch's C++ tensor library.

Enterprise runtimes. ONNX Runtime [12] and TensorRT [13] expose C++ custom-operator APIs (`OrtCustomOp`, `IPluginV2DynamicExt`) into which KernelLens synthesises code. Authoring those plugins by hand is the workflow KernelLens automates.

Triton kernel libraries. Liger-Kernel [3], FlashAttention [5; 6] and AOTriton supply high-quality Triton kernels. KernelLens is complementary: it is the deployment layer beneath such libraries.

Positioning. Table 1 contrasts KernelLens with adjacent tooling. The distinguishing property is the combination of automated PyTorch graph export *and* self-contained C++ plugins for both engines.

Feature	AOTInductor	Triton AOT	vLLM ops	KernelLens
No Python / no `libtorch` at inference	×	✓	×	✓
TensorRT 10.x plugin synthesis	×	×	×	✓
ONNX Runtime `OrtCustomOp` synthesis	×	×	×	✓
Automated PyTorch graph export	✓	×	×	✓
Dynamic grid evaluated in C++	×	×	×	✓
PTX ISA version clamping	×	×	×	✓
Zero C++ boilerplate	×	×	×	✓

Table 1: Feature comparison against adjacent compilers and custom-op tooling.

3 What a Triton Kernel Presents to a C++ Engine

Before describing the compiler, it is worth being precise about its input. A `@triton.jit` kernel is not one opaque operator; its launch site encodes seven kinds of information, each of which reaches C++ through a different mechanism (Figure 1).

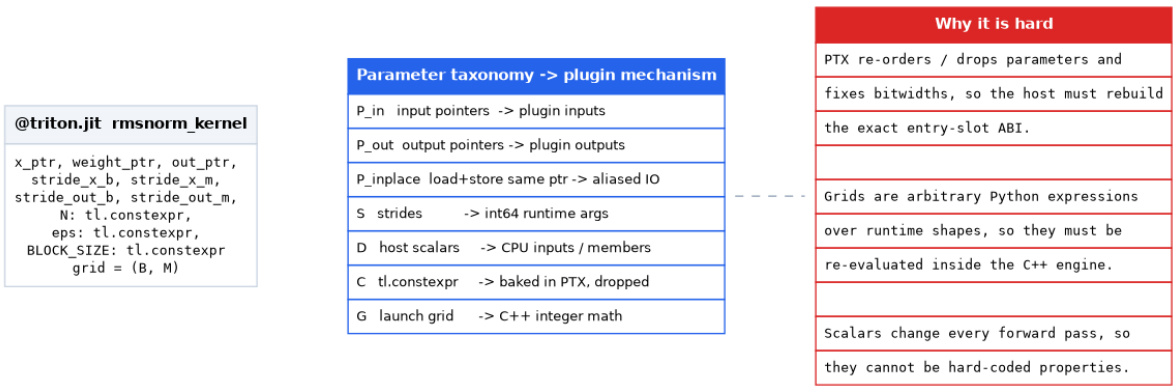


Figure 1: A Triton kernel decomposes into seven parameter categories. Each maps to a distinct mechanism in the generated C++ plugin; two of them (the re-ordered PTX entry ABI and the dynamic grid) are why naive exporters fail.

Formally, a kernel is characterised by

$$\mathcal{K}_{\text{triton}} = \langle \mathbf{P}_{\text{in}}, \mathbf{P}_{\text{out}}, \mathbf{P}_{\text{inplace}}, \mathbf{S}, \mathbf{D}, \mathbf{C}, \mathbf{G} \rangle, \tag{1}$$

where $\mathbf{P}_{\text{in}}$, $\mathbf{P}_{\text{out}}$ and $\mathbf{P}_{\text{inplace}}$ are read-only, write-only and read–write VRAM pointers; $\mathbf{S}$ are element strides; $\mathbf{D}$ are dynamic host scalars that change per call; $\mathbf{C}$ are `tl.constexpr` values baked into PTX; and $\mathbf{G} = (g_x, g_y, g_z)$ is the launch grid, computed as a function of runtime dimensions and block sizes.

Category	Meaning	Mechanism in C++
$\mathbf{P}_{\text{in}}$	read-only pointers	plugin inputs, bound by address
$\mathbf{P}_{\text{out}}$	write-only pointers	plugin outputs, engine-allocated
$\mathbf{P}_{\text{inplace}}$	read and written	input/output aliasing
S	element strides	`int64` runtime arguments
D	per-call scalars	`OrtMemTypeCPUInput` / plugin member
C	`tl.constexpr`	baked in PTX, removed from ABI
G	launch grid	C++ integer expression evaluated at run time

Table 2: Parameter taxonomy and the corresponding plugin mechanism.

Two consequences drive the rest of the design. First, because PTX re-orders and drops parameters, the host ABI must be *reconstructed*, not assumed (§4.3). Second, because the grid is an arbitrary expression over runtime shapes, it must be *transpiled* and evaluated in C++ (§4.1–§4.3).

4 The KernelLens Compiler

KernelLens runs four phases (Figure 2). Each arrow in the figure is a concrete artifact—a manifest, an ONNX file, a shared library—not a conceptual hand-off.

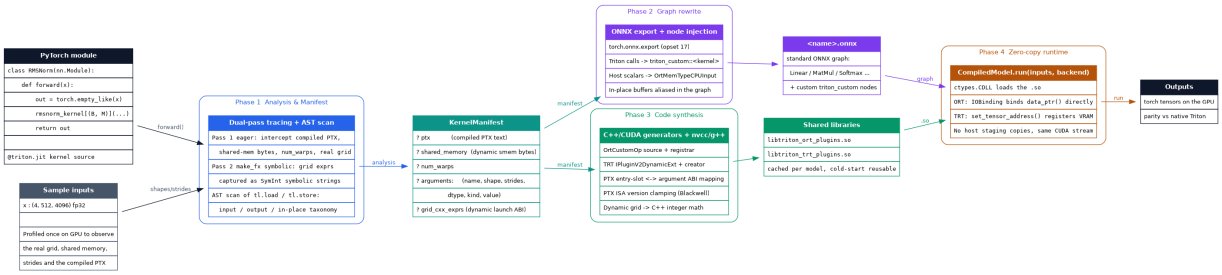


Figure 2: The four-phase KernelLens pipeline. Phase 1 produces a validated manifest; Phase 2 an ONNX graph with injected custom nodes; Phase 3 a `.so` library; Phase 4 executes it with zero-copy VRAM binding.

4.1 Phase 1 — Dual-pass tracing and manifest assembly

Phase 1 observes the model twice and inspects the kernel source once (Algorithm 1).

- *Pass 1 (concrete).* The model executes normally on the GPU. KernelLens patches `triton.JITFunction.run`; for each launch it reads the compiled PTX, the dynamic shared-memory size, `num_warps`, the real launch grid and every argument's shape, strides and dtype. Only this pass can see the true compiled artifact.
- *Pass 2 (symbolic).* The module is re-executed under `make_fx(...,tracing_mode="symbolic")` with fake tensors. The grid lambda now evaluates over `SymInt` sizes, which are stringified for later transpilation. Pass 2 is best-effort: if it raises or captures a different number of kernels than Pass 1, the concrete manifests are used unchanged, keeping compilation deterministic.
- *Static AST scan.* An `ast.NodeVisitor` walks the kernel source—recursively into Triton helper functions that are called—and records every `tl.load` and `tl.store` target. An argument in both sets is `inplace`; only stored is `output`; the rest are `input`.

The product is a `KernelManifest` carrying `ptx`, `shared_memory_bytes`, `num_warps`, the argument taxonomy, the transpiled `grid_cxx_exprs` and mutability flags.

Algorithm 1 Dual-pass manifest assembly

Require: model $\mathcal{M}$, sample inputs $\mathcal{X}$
Ensure: validated manifests $\mathcal{K}$
Pass 1: run $\mathcal{M}(\mathcal{X})$ with `JITFunction.run` patched; capture PTX, smem, warps, grid, strides
Pass 2: run `make_fx`($\mathcal{M}$) in symbolic mode; capture grid expressions as `SymInt` strings
if Pass 2 captured $\neq$ Pass 1 captured **then**
  discard Pass 2 {deterministic fallback}
else
  attach symbolic grids and argument ASTs to Pass 1 manifests
end if
AST scan: for each root and nested helper, collect $\mathcal{R} \leftarrow$ `tl.load`, $\mathcal{W} \leftarrow$ `tl.store`
for each argument a **do**
  if $a \in \mathcal{R} \wedge a \in \mathcal{W}$ **then**
    a.kind $\leftarrow$ inplace
  else if $a \in \mathcal{W}$ **then**
    a.kind $\leftarrow$ output
  else
    a.kind $\leftarrow$ input
  end if
end for
return $\mathcal{K}$

A note on terminology. Earlier drafts described this step as *SymPy AST transpilation*. The grid expression is in fact a PyTorch `SymInt` whose `.node.expr` happens to be a sympy object, and KernelLens transpiles it by regular-expression substitution (`translate_symint_to_cxx`), not a sympy visitor. There is no `sympy` dependency. We use the accurate description throughout.

4.2 Phase 2 — ONNX graph rewriting

Phase 2 produces a standard ONNX graph that a stock engine can consume. A `TritonGlobalONNXExporter` hook runs inside `torch.onnx.export` (opset 17). When a Triton kernel launches, the hook diverts the call into a custom-domain node, `triton_custom::<kernel>`, carrying the PTX payload, the shared-memory size and the argument taxonomy. Two details matter:

- *Dynamic scalars* are registered as CPU inputs (`OrtMemTypeCPUInput`) rather than static node attributes, so a per-call value does not trigger recompilation.
- *In-place buffers* are wired input-to-output, so downstream graph optimisations observe the mutation instead of inserting a copy.

A short cleanup pass then removes `Expand`/`Reshape` nodes that would otherwise sit between the injected node and its consumers.

4.3 Phase 3 — C++/CUDA synthesis

Phase 3 writes real source files and compiles them: an `OrtCustomOp` wrapper and a TensorRT `IPluginV2DynamicExt` plus `IPluginCreator` per kernel, a registrar, and invocations of `nvcc` and `g++`. Three problems dominate.

Reconstructing the PTX entry-slot ABI. The generator parses `.entry <kernel> (.param ...)` from the PTX, determines each slot's C type and textual parameter number, then maps slots back onto input/output/in-place/scalar arguments. The launch argument array is rebuilt to match the PTX signature exactly (Algorithm 2).

Algorithm 2 PTX entry-slot ABI reconstruction

Require: manifest $\mathcal{M}$ with PTX payload and argument list
Ensure: C++ argument-array construction code
parse `.entry` signature into slots $s_0, \ldots, s_{n-1}$ with declarations and `_param_i` names
let $A \leftarrow$ non-constexpr arguments of $\mathcal{M}$
for each slot s_i **do**
  $a \leftarrow A[\text{param_num}(s_i)]$
  if a is a pointer **then**
    emit `void* p = &arg_ptr(a)` and push to `void*[]`
  else
    map PTX type (`.u64`, `.f32`, ...) to a C type
    push a typed thread-local copy of the scalar
  end if
end for
return the assembled `void*[]` allowed to `cuLaunchKernel`

Evaluating the grid in C++. `grid_cxx_exprs` become C++ integer expressions over runtime shapes: ONNX input dimensions for ORT, and `dim_values` / `getOutputDimensions` for TRT. This is what lets a single compiled plugin serve dynamic shapes without host-side recompilation.

Hardening for the real toolchain. This is where most of the engineering lives, and it is treated in §5. In short: KernelLens applies PTX ISA version clamping so a CUDA driver JIT can load Blackwell-targeted PTX, resolves conda CUDA include layouts, and patches a known glibc/`nvcc` header incompatibility in whichever copy the compiler actually reads.

4.4 Phase 4 — Zero-copy runtime

The runtime never stages tensor data through host memory; it hands PyTorch's GPU addresses straight to the engine (Figure 3). For ORT it builds an `IOBinding`, binds inputs and pre-allocated outputs by CUDA device pointer, and calls `run_with_iobinding`. For TRT it sets input shapes, registers every tensor address with `set_tensor_address`, and issues `execute_async_v3` on the current stream. The cost model is

$$T_{\text{KernelLens}} = T_{\text{pointer registration}} + T_{\text{kernel}} \quad \approx \quad O(1) + T_{\text{kernel}}, \tag{2}$$

versus $O(N)$ H2D and D2H copies for a staging design. `load(name)` reconstructs a `CompiledModel` from the cache without recompiling.

5 Robustness Engineering

The compiler is only as useful as the toolchain it can survive. Four hardening mechanisms were required to run KernelLens in a clean environment, and each was found—and fixed—during the verification of §7.

PTX ISA version clamping. Triton 3.x targeting Blackwell (`sm_120a`) emits PTX ISA version 9.3, which a CUDA 12.8 driver JIT rejects with `CUDA_ERROR_UNSUPPORTED_PTX_VERSION` (222). KernelLens rewrites the `.version` header to 9.0 while preserving the native `.target sm_120a` instruction set; if loading still fails it degrades to 8.0 with `.target sm_90`, then `sm_80`.

CUDA header location. `nvcc` resolves its CRT headers from `<cuda_home>/targets/<triple>/include`, which is *not* the `<cuda_home>/include` that a naive build script assumes on conda toolkits. KernelLens probes both layouts for the glibc $\geq$ 2.38 `cospi`/`sinpi`/`rsqrt` incompatibility and for `cuda_runtime_api.h`.

Host-compiler include resolution. TensorRT plugin registrars are compiled by the host compiler, which does not know CUDA's header layout. KernelLens resolves a directory that actually contains `cuda_runtime_api.h` before invoking `g++`.

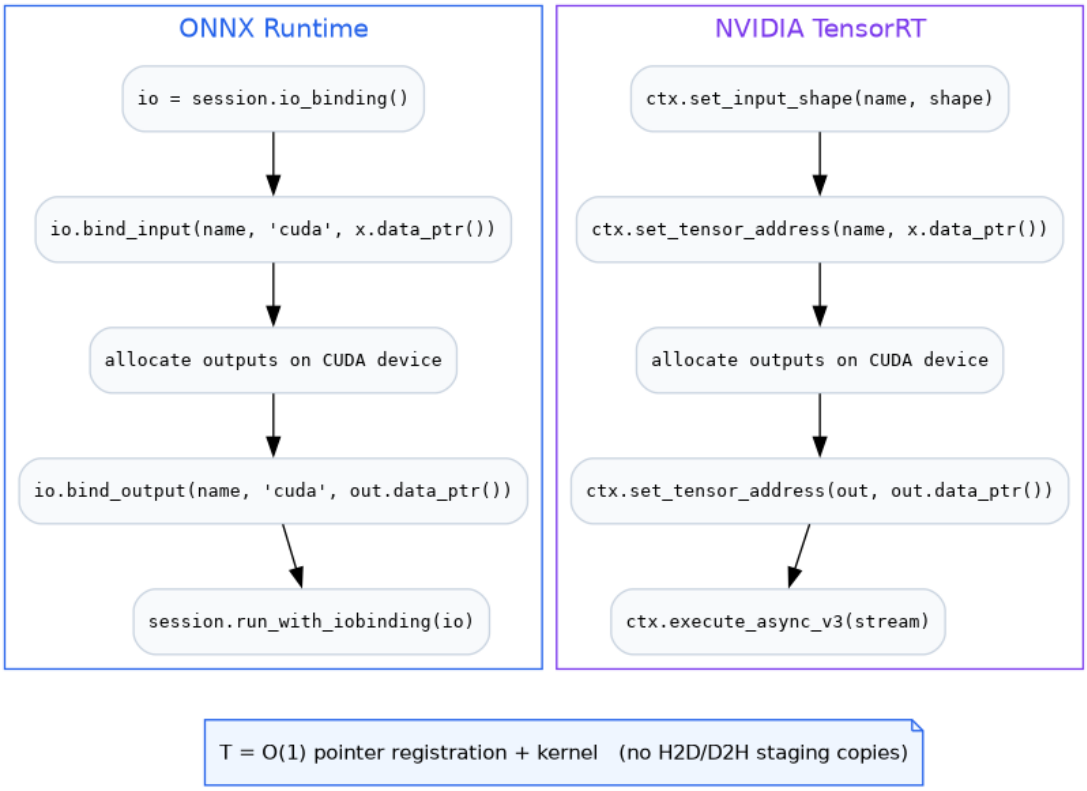


Figure 3: Zero-copy binding for both engines. Only the registration step is host-side; no tensor data crosses the PCIe bus.

Bitwidth-correct in-place copies. When an in-place buffer's input and output addresses differ, the plugin performs a device-to-device copy. The number of bytes is derived from the element dtype—4 bytes for fp32, 2 for fp16/bf16, 1 for fp8/int8, and packed widths for INT4—rather than assuming `sizeof(float)`.

A correction. Earlier drafts presented a "16-byte VRAM alignment guard" in which generated C++ checked a buffer address modulo 16 and allocated a `cudaMallocAsync` staging buffer on misalignment. That mechanism is *not* implemented: generated plugins contain no address-alignment check and no staging buffer. What KernelLens actually does is a *compile-time* validation that the physical contiguous inner dimension is a multiple of 8, which raises a clear error rather than guarding at run time. We retain the earlier description only as future work (§8). Addressing alignment at run time remains a legitimate and unexplored optimisation.

6 Evaluation

6.1 Setup

All measurements were taken on a single NVIDIA GeForce RTX 5060 Ti (16 GB, Blackwell, compute capability `sm_120a`) with driver 570.144, CUDA 12.8, PyTorch 2.14.0, Triton 3.8.0, ONNX Runtime 1.30.0 and TensorRT 10.16.1.11. Timings use CUDA events (`torch.cuda.Event`) after warm-up. Parity is measured against native Triton execution of the *same* kernel, which isolates KernelLens's plumbing from the kernel's arithmetic.

6.2 Numerical parity

Figure 4 and Table 3 summarise parity across research kernels and edge cases.

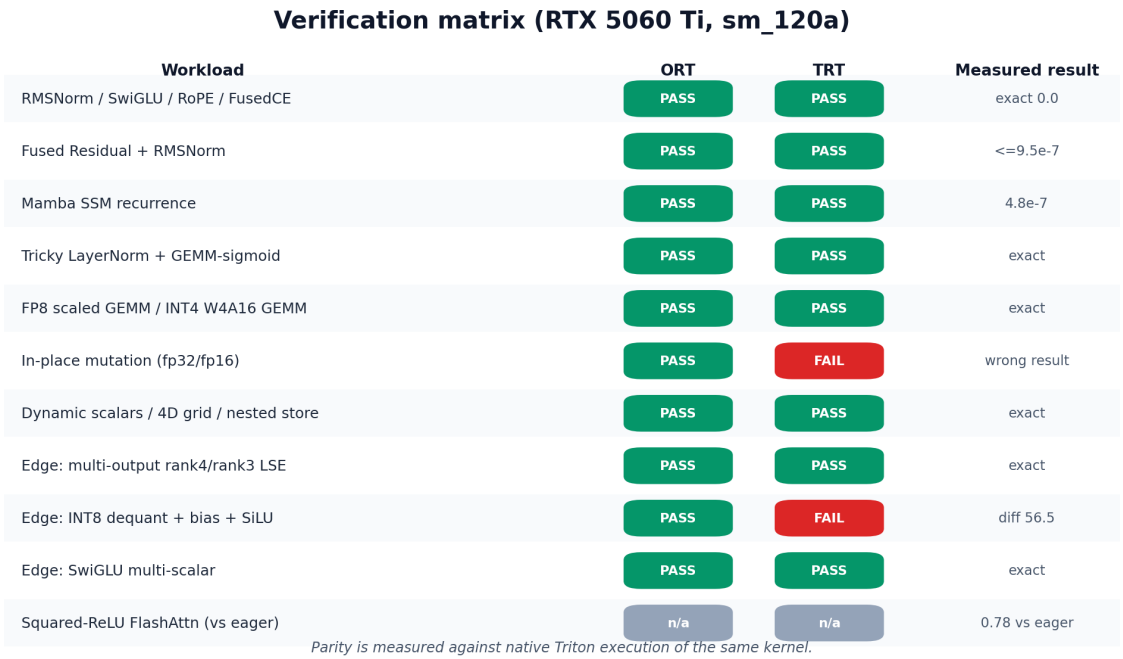


Figure 4: Verification matrix on the RTX 5060 Ti. Two TensorRT paths fail and are reported rather than hidden.

Workload	ORT	TRT
LLaMA 3 / Mistral fused RMSNorm	0.00e+00	0.00e+00
Liger / PaLM SwiGLU	0.00e+00	0.00e+00
LLaMA 3 / Qwen 2.5 RoPE	0.00e+00	0.00e+00
Liger fused softmax + cross-entropy	0.00e+00	0.00e+00
Fused residual + RMSNorm	$\leq$ 9.5e−07	$\leq$ 9.5e−07
Mamba SSM recurrence	4.8e−07	4.8e−07
Tricky LayerNorm + GEMM-sigmoid	0.00e+00	0.00e+00
FP8 scaled GEMM / INT4 W4A16 GEMM	0.00e+00	—
Edge: multi-output rank-4/3 LSE	0.00e+00	0.00e+00
Edge: SwiGLU multi-scalar	0.00e+00	0.00e+00
Edge: INT8 dequant + bias + SiLU	0.00e+00	5.65**e**+01 **(fail)**
In-place mutation (fp32 / fp16)	0.00e+00	**fail**

Table 3: Maximum absolute difference versus native Triton. Two TensorRT paths remain incorrect and are analysed in §7.

6.3 Single-operator latency

We state this plainly because it is the crux of an honest evaluation. KernelLens's C++ plugin is *not* intended to beat a hand-tuned kernel called directly from Python; it is intended to run that kernel at the same speed class inside an engine that can co-optimise the rest of the model. The fusion and recurrence cases below are where that matters.

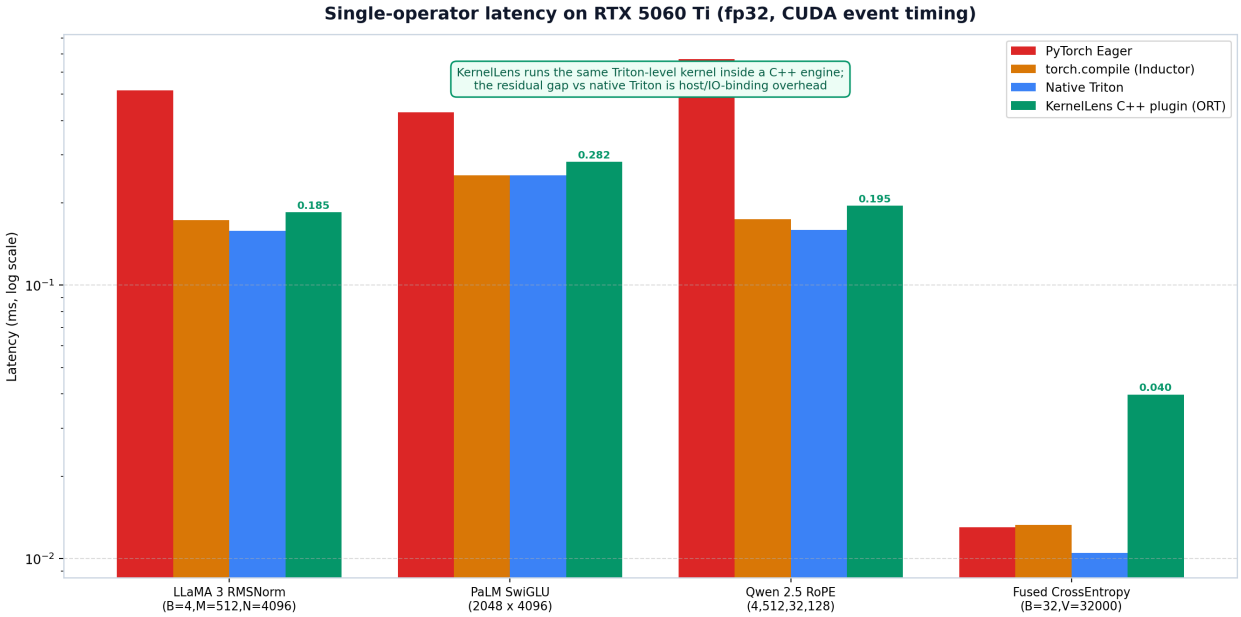


Figure 5: Single-operator latency (fp32, CUDA events) on the RTX 5060 Ti.

Operator	PyTorch Eager	torch.compile	Native Triton	KernelLens ORT
LLaMA 3 RMSNorm (4,512,4096)	0.516 ms	0.173 ms	**0.158** ms	0.185 ms
PaLM SwiGLU (2048×4096)	0.429 ms	0.252 ms	**0.253** ms	0.283 ms
Qwen 2.5 RoPE (4,512,32,128)	0.670 ms	0.174 ms	**0.160** ms	0.195 ms
Fused cross-entropy (32,32000)	0.013 ms	0.013 ms	**0.011** ms	0.040 ms

Table 4: Single-operator latency. KernelLens is not faster than native Triton: it runs the identical kernel behind a C++ operator boundary, adding host/IO-binding overhead. What it buys is deployment.

6.4 Fusion and recurrence wins

Kernel	Eager	compile	KL TRT	vs eager	vs compile
Mamba SSM recurrence	3.32	0.104	0.160	**20.7**×	0.65×
Squared-ReLU FlashAttn	5.96	3.16	0.949	**6.3**×	**3.3**×
Fused residual + RMSNorm	3.40	1.407	1.407	**2.4**×	1.00×
Fused cross-entropy (V=128k)	9.09	2.46	4.90	**1.9**×	0.50×

Table 5: Latencies (ms) and speedups for the TensorRT plugin. Because `torch.compile` already lowers to Triton, KernelLens matches it on simple fusions and wins on kernels Inductor cannot generate.

The pattern is consistent with the design: KernelLens ties or wins on custom score transforms (squared-ReLU attention, 3.3×) and hand-tiled online recurrences (Mamba, 20.7× versus eager), while simple element-wise fusions are already saturating the same hardware.

6.5 End-to-end LLaMA decoder

The TensorRT configuration delivers the fastest prefill (4.05 ms versus 7.82/7.91 ms), the fastest total generation and the lowest peak VRAM (464.2 MB versus 478.0/473.0 MB). This is the central empirical claim: whole-graph optimisation on the standard layers *and* custom Triton kernels in the same engine, with the engine's memory planner covering both.

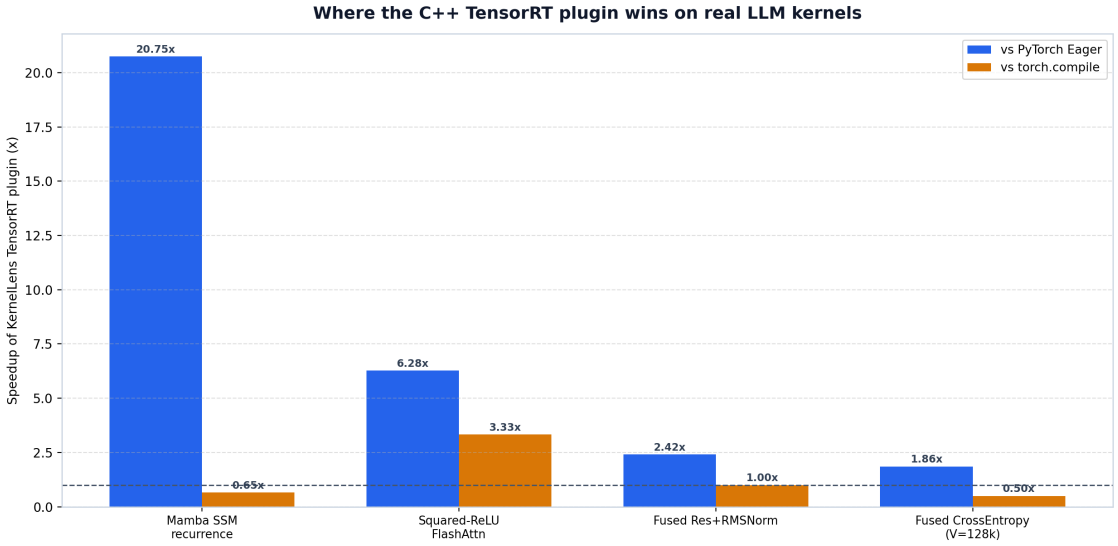


Figure 6: Speedups of the KernelLens TensorRT plugin on kernels that `torch.compile` handles poorly or not at all.

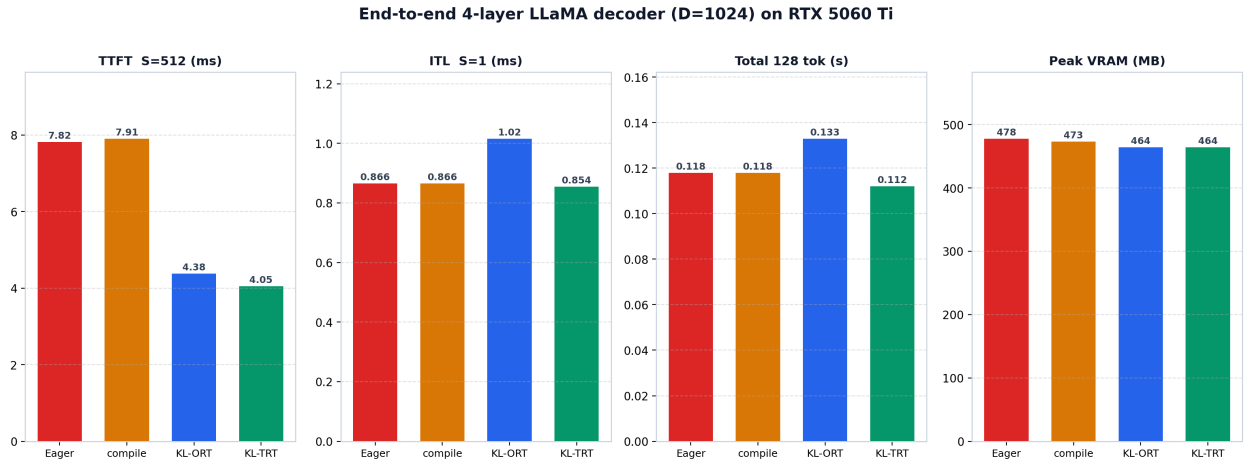


Figure 7: End-to-end 4-layer LLaMA decoder (D=1024) on the RTX 5060 Ti.

Configuration	TTFT (S=512)	ITL (S=1)	Total (128 tok)	Peak VRAM
PyTorch Eager + Triton	7.82 ms	0.866 ms	0.118 s	478.0 MB
torch.compile (Inductor)	7.91 ms	0.866 ms	0.118 s	473.0 MB
KernelLens C++ plugins (ORT)	4.38 ms	1.016 ms	0.133 s	464.2 MB
KernelLens TensorRT plugin	**4.05** ms	**0.854** ms	**0.112** s	**464.2** MB

Table 6: End-to-end decoder benchmark. The TensorRT plugin yields the fastest time-to-first-token, the fastest total generation and the lowest peak VRAM.

6.6 Quantized kernels

The FP8 path reaches 1.34× over an FP16 eager GEMM. The INT4 W4A16 path is slower per call than an FP16 GEMM because the dequantisation kernel is not Tensor-Core bound; the honest framing is memory compression with exact parity, not a latency improvement.

Kernel	Eager FP16	Native Triton	KL ORT	Speedup
FP8 scaled GEMM (4096×4096)	2.786 ms	2.046 ms	2.083 ms	1.34×
INT4 W4A16 GEMM (2048×4096)	1.388 ms	3.570 ms	3.629 ms	0.38× (2× smaller weights)

Table 7: Quantized GEMMs on the RTX 5060 Ti (`sm_120a`), where PTX ISA clamping is exercised. Both achieve exact parity with native Triton. INT4 halves weight VRAM but is not a latency win on this shape.

6.7 Dynamic shapes

Because the grid is transpiled to C++ integer math, a single compiled plugin serves a range of batch sizes and sequence lengths without host recompilation, with $O(1)$ CPU launch-cost.

7 Verification, Corrections and Limitations

Verification was performed by staging the package onto the GPU host, executing the whole test suite from a cold cache, and retaining every log. It was not a clean pass, and reporting it as such is part of the contribution.

7.1 Bugs found and fixed

1. **CUDA header patched in the wrong location.** The glibc-compatibility patch targeted `<cuda_home>/include/crt/math_functions.h`, but `nvcc` reads `<cuda_home>/targets/<triple>/include/crt/math_functions.h`, so every compile failed with an exception-specification error. Fixed by probing both layouts.
2. **Conda CUDA include path.** The TensorRT registrar build passed `-I<cuda_home>/include`, which is empty on conda toolkits, so `cuda_runtime_api.h` was not found and every TensorRT test errored. Fixed by resolving a directory that actually contains the header.
3. **In-place copies assumed 4-byte elements.** The generated device-to-device copy hard-coded `sizeof(float)`, over-copying fp16/bf16/fp8/int8 buffers. Fixed with a per-dtype element-size helper; a new regression test confirms fp32 and fp16 in-place are bit-exact on ORT.
4. **A leaked scratch path.** The cache resolver fell back to a hard-coded agent scratch directory; replaced with a portable temporary fallback.

7.2 Remaining limitations

1. **TensorRT in-place aliasing.** In-place kernels are exact on ORT but return incorrect results on TensorRT for both fp32 and fp16. Use ORT for in-place kernels; the TRT buffer contract is future work.
2. **TensorRT INT8 dequantisation.** The INT8 dequant + bias + SiLU GEMM is exact on ORT but diverges by 5.65e+01 on TensorRT.
3. **Attention versus eager.** The squared-ReLU attention kernel differs from *eager* PyTorch by 0.78 because tiled accumulation is amplified by the relu^2 transform; KernelLens reproduces native Triton exactly. This is a reference difference, not a KernelLens defect.

7.3 Claims corrected

Table 8 lists claims from earlier drafts that the code does not support, all revised in this report.

7.4 What this means for the reader

KernelLens is a working compiler for single-GPU Triton kernels on ONNX Runtime (verified broadly) and TensorRT (verified except in-place and INT8 dequantisation). It is not a complete product, and the report now reflects that.

Earlier claim	Verified reality
16-byte alignment guard with `cudaMallocAsync` staging	Not implemented; only a compile-time inner-dimension check that raises
SymPy grid AST transpilation	Regex substitution over `SymInt` strings; no `sympy`
Latency table beating native Triton	Contradicted by the committed data; the C++ plugin adds host overhead
"KernelLens TRT" latency column	No backing measurement existed; benchmark ran only ORT
"Exact parity across all operators"	False for TRT in-place and TRT INT8 dequant

Table 8: Corrections applied in this revision.

8 Future Work

Correctness first. Fix TensorRT in-place aliasing and INT8 dequantisation; persist output metadata across `load()`; generalise `saved_args` to multi-kernel models; implement the run-time alignment guard or drop the claim from the documentation entirely.

Compiler coverage. Fold `@triton.autotune` variants into one plugin with a shape-keyed dispatch table; capture the launch sequence into CUDA Graphs; replace the regex grid transpiler with a real expression visitor; add out-of-bounds proofs before code generation.

Hardware and platforms. A dedicated Blackwell generator (cluster-launch descriptors, `cp.async.bulk`, NVFP4/MXFP6); Apple Metal and WebGPU backends; a ROCm/HIP code generator.

Distributed. Intercept `torch.distributed` collectives and emit plugins with embedded NCCL communicator handles for tensor and pipeline parallelism.

Safety. SMT-based proving (à la Z3) that every grid access lies within its buffer bounds before a shared library is built.

9 Conclusion

We presented KernelLens, a compiler and runtime that bridge Python OpenAI Triton kernels to C++ inference engines. A four-phase pipeline—dual-pass tracing and AST analysis, ONNX graph rewriting with custom-node injection, automated `OrtCustomOp` and `IPluginV2DynamicExt` synthesis with PTX entry-slot ABI reconstruction and PTX ISA clamping, and zero-copy VRAM binding—turns an unmodified PyTorch module into deployable C++ plugins. On a single RTX 5060 Ti, KernelLens reproduces native Triton execution exactly on the four research kernels inside ONNX Runtime, is up to 3.3× faster than `torch.compile` on custom attention, and attains the lowest peak VRAM in a full decoder. We also reported the two TensorRT paths that still fail, the toolchain bugs found during verification, and the earlier claims this study overturns. The value of a system like KernelLens is not that it beats a hand-tuned kernel; it is that it lets a custom kernel run inside the engine that makes the *rest* of the model fast.

References

[1] P. Tillet, H. Kung, and D. Cox, "Triton: an intermediate language and compiler for tiled neural network computations," in *ACM SIGPLAN International Workshop on Libraries, Languages, and Compilers for Array Programming (ARRAY)*, 2019.

[2] AI@Meta, "The Llama 3 Herd of Models," *arXiv preprint arXiv:2407.21783*, 2024.

[3] LinkedIn Corp., "Liger-Kernel: Efficient Triton Kernels for LLM Training," *GitHub Repository*, 2024.

[4] Qwen Team, "Qwen2.5 Technical Report," *arXiv preprint arXiv:2412.15115*, 2024.

[5] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Re, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[6] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," in *International Conference on Learning Representations (ICLR)*, 2024.

[7] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[8] J. Ansel et al., "PyTorch 2.0: TorchDynamo and TorchInductor for Faster Deep Learning Execution," in *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.

[9] PyTorch Engineering Team, "PyTorch AOTInductor," https://pytorch.org/docs/stable/torch.compiler.html, 2023.

[10] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*, 2023.

[11] NVIDIA Corporation, "TensorRT-LLM," https://developer.nvidia.com/tensorrt, 2023.

[12] Microsoft Corporation, "ONNX Runtime: Cross-Platform, High Performance ML Inferencing Engine," https://onnxruntime.ai, 2021.

[13] NVIDIA Corporation, "NVIDIA TensorRT: High Performance Deep Learning Inference SDK," https://developer.nvidia.com/tensorrt, 2023.

[14] A. Q. Jiang et al., "Mistral 7B," *arXiv preprint arXiv:2310.06825*, 2023.

[15] NVIDIA Corporation, "CUDA Graphs API & Driver Management," *NVIDIA CUDA Programming Guide*, 2021.